OPENCTI / PEER REVIEW

Modernization and implementation

Phase 1-3 delivery plan
Evidence, ownership and acceptance

Updated November 13, 2026

Included in this revision	Review focus
Part I - modernization roadmap	Preserved adoption/debugging contracts, Git-primary/SVN policy and current Phase 0 evidence/status
Part II - detailed implementation	19 proposed pull requests with dependencies, accountable roles, acceptance checks and rollback
Reference baseline	PR #17 / c6e06a8: measured legacy histories, retained FT evidence, proposed schemas and ten discrepancies
Release gates G1-G3	Native browsing, recoverable Docker execution and opt-in native campaign cutover

Planning status: Phase 0 technical preparation is ready for peer review; formal creator signoff and PR #17 merge remained pending when this revision was prepared. Implementation milestones are proposed work. Named human ownership and production route approvals must be recorded.

Contents

Direction and fixed requirements . . . 4
Target structure . . . 5
Language and storage choices . . . 6
Dashboard continuity is the first delivery track . . . 6
Git as primary repository with SVN coexistence . . . 8
Suite adoption and script preservation . . . 9
Contracts that permit incremental replacement . . . 12
Coexistence and rollback . . . 13
Docker execution with Mac-native evidence . . . 14
Failed-test debugging environments are a core product contract . . . 14
Remote GPU execution and return of results . . . 17
New-language and Open64 DSL testing . . . 18
Delivery sequence and acceptance gates . . . 18
Phase 0 working artifacts . . . 20
First implementation scope . . . 21
Part II - Phase 1-3 implementation plan . . . 22
Decision and evidence baseline . . . 22
Ownership and working rules . . . 23
Product boundaries and proposed interfaces . . . 24
Milestone dependency map . . . 25
Phase 1 - native results and evidence dashboard . . . 26
P1-01 - freeze contracts and bootstrap the native service . . . 26

P1-02 - implement SQLite persistence and artifact transactions . . . 26
P1-03 - import legacy evidence and expose bounded queries . . . 26
P1-04 - build read-only campaign and failure browsing . . . 27
P1-05 - expose failure preservation and existing session access . . . 27
P1-06 - qualify the Phase 1 slice and hand it over . . . 27
Phase 2 - Docker execution and recoverable debugging . . . 28
P2-01 - introduce a bounded local Docker backend . . . 28
P2-02 - snapshot sources, baselines and mounted dependencies . . . 28
P2-03 - make dispatch and cancellation durable . . . 28
P2-04 - collect verified artifacts with retry and retention rules . . . 28
P2-05 - integrate portable debugging sessions . . . 29
P2-06 - execute the destructive Phase 2 recovery gate . . . 29
Phase 3 - native harness and Git-primary adoption . . . 29
P3-01 - resolve native catalogs and effective options . . . 29
P3-02 - implement native phase execution and verdict semantics . . . 30
P3-03 - implement the regression adapter . . . 30
P3-04 - implement the application wrapper adapter . . . 30
P3-05 - implement the external script-suite adapter . . . 30
P3-06 - add reviewed Git adoption transactions and SVN routing . . . 31
P3-07 - qualify campaign routing and reversible cutover . . . 31
Discrepancy and coverage commitments . . . 32
Acceptance records, releases and operational limits . . . 33
Evidence references and next action . . . 33

Direction and fixed requirements

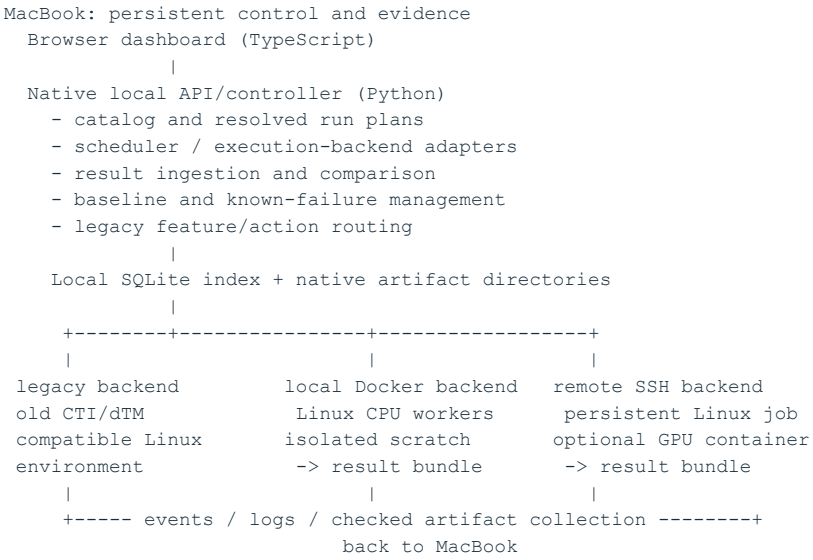
Preserving a usable failed-test debugging environment is the central compatibility requirement of this modernization. Modernize CTI incrementally around a native MacBook dashboard and local evidence store. Retain the existing framework as an executable compatibility path while replacing modules behind explicit contracts. The final harness and dashboard should not require Perl; keeping the old runtime during migration is a temporary compatibility measure, not the target architecture.

This is a proposal, not an implementation. It follows the architecture reconstruction and the requirements below. Phase gates describe evidence required before switching a component; they do not claim the current legacy installation has already been validated.

Requirement	Architectural commitment
Preserve CTI failure triage	Retain sources, harness dependencies, compile/run/compare scripts, original baseline, workspace evidence, and a replayable container/mount specification for each failed attempt
Retire Perl	Python for orchestration/adapters and a TypeScript web UI; limited shell for transparent tool invocation; replace Perl modules by responsibility
Keep old framework operational while swapping modules	Parallel legacy execution path, versioned contracts, per-campaign routing, and reversible cutovers
Preserve dashboard features throughout migration	Feature inventory and acceptance fixtures first; retain working legacy views/actions until equivalent native workflows pass
Preserve suite-adoption scripts and workflows	Retain regression discovery, application wrappers, external script-suite adapters, specialized hooks and legacy enrollment tools until their replacement workflows pass explicit acceptance gates
Make Git the primary source repository tool	Use Git for OpenCTI development, suite integration and reviewed source/configuration/master changes; retain SVN through an explicit legacy repository adapter with one authoritative repository per source scope
Add languages, including the Open64 DSL	Language/toolchain adapters and phase-based test descriptions, independent of filename extension and execution host
Execute in Docker, keep dashboard and baseline artifacts native to MacBook	Native local service/storage, isolated Linux workspaces, explicit result collection and baseline versioning
Submit to remote GPU hosts, keep results locally	SSH-based submission, persistent remote job state, resumable artifact transfer, and local ingestion; no shared filesystem requirement

Interpret “native” as a service and files running directly on macOS, with the dashboard viewed in a browser. An AppKit/Swift desktop application is not required for this initial design. Remote GPU work is assumed to target a supported Linux machine, with its actual GPU vendor/runtime detected before configuration. The DSL's commands, IR formats, and correctness oracles remain adapter requirements to establish from the Open64 project.

Target structure



Keep one deployable local control service initially, with internal module interfaces rather than separate network services for every responsibility. The legacy backend may execute in a compatible Linux container/host; the Mac does not need a native Perl CGI installation to browse the new dashboard. dTM remains available for campaigns still using it, and can retire after the replacement backend covers the pools and reservation features actually in use.

The Mac service owns the database and cataloged baseline versions. Workers receive immutable input snapshots, write scratch/output data, and return result bundles. Workers cannot directly update approved baselines or known-failure annotations.

Language and storage choices

Responsibility	Proposed choice	Reason
CLI, controller, importers, toolchain adapters, workers	Python	Suitable for process/file orchestration, structured manifests, numerical validation integration, and the existing Python-oriented Open64 work
Dashboard	TypeScript	Typed result/action contracts and maintainable browser interactions for historical comparisons and large result tables
Small environment/bootstrap wrappers	Shell where useful	Preserve readable tool commands without moving scheduling, state, or verdict logic into shell
Metadata index	SQLite, accessed only by the native service	Local persistence and query support without running a database server
Logs, sources, masters, IR, binaries, measurements	Native content-addressed files plus manifests	Keep large/binary evidence outside database rows; preserve provenance and original logical filenames
Compatibility runtime	Existing Perl/shell/Java/PHP in a legacy Linux environment	Keep existing features working during component-by-component replacement

Select concrete web/API frameworks during the first dashboard slice, using feature and table-size tests; the architecture does not depend on an early framework choice. Version public schemas and adapter protocols before introducing framework-specific APIs. Python's subprocess API provides explicit arguments, environments, working directories, timeouts, output capture, and return codes; process-tree cancellation still needs an execution-backend contract.

SQLite remains on the Mac's local disk. Remote workers and containers exchange files/events with the service, not direct database writes. If WAL mode is selected, keep all database access on that same host; SQLite documents that WAL does not work across network filesystems. See SQLite WAL constraints.

Dashboard continuity is the first delivery track

Establish the compatibility inventory from the actual CGI/PHP implementation, not just the rewritten guides. Treat viewing, triage, and administrative actions separately so a read-only screen is not mistaken for full parity.

Feature to preserve	Existing implementation evidence	Migration acceptance
Schedule/options-by-day matrix, totals, setup/running/completed/old-log states	`show-schedule.cgi`, `TestInfo.pm`	Same fixture yields matching counts/states; options and launcher logs remain accessible

Feature to preserve	Existing implementation evidence	Migration acceptance
Failure details, categories, filters, include/exclude buckets, sorting and display limits	`show-failures.cgi`, `chk_test_errors.cgi`	Same test identities and failure classes remain findable; truncation is visible
Previous-run comparisons and newly changed failures	`show-failures.cgi`, `show_regressions.pl`	Matching rules and reference-run choice reproduce characterized examples
Known failures: comments, dates, same-message/pattern expansion, deletion, import, and text/grouped reports	Known-failure CGI tools, `bucketize.cgi`, cache helpers	Annotation identity/meaning survives import; edits do not change raw verdicts
Preserved failure workspace and independent compile/run/compare replay	Existing driver environment files, generated scripts, source links and retained failure work trees	Open a reconstructed workspace after original container deletion; run each phase against the original baseline; preserve original evidence
Compile/runtime/triage logs, generated scripts, options and expected-versus-actual output	`get-*-file.cgi`, `get-build-log.cgi`, `chk_test_errors.cgi`	A failed test can be investigated from the dashboard with original evidence
Rerun preview/launch, rerun history, test launches from schedules	`rerun_test.cgi`, `start_tests.cgi`	Selected configuration/test is reproduced and linked as a new attempt
Remaster preview/application and test enrollment	`remaster-test.cgi`, `addtest.cgi` and helpers	Original operations remain available; native replacement records what changed and who initiated it
Elapsed/cumulative time, version/context display, menus and links	Schedule/failure CGI parameters	Relevant context and navigation are retained
Users, jobs, pools, machine status, enable/disable, cancellation, performance reservations	`dTMState.php`, `dtmcmd.cgi`, dTM commands	Legacy campaigns retain controls; new jobs expose equivalent supported actions

This is the initial code-derived inventory; phase 0 must enumerate remaining CGI endpoints/options and turn used behaviors into explicit fixtures. Do not promise pixel-identical rendering or identical defects. Preserve feature semantics and access to original evidence; document intentional bug fixes individually.

The first native dashboard release should view imported legacy results on the Mac. Until each native workflow reaches parity, its legacy view/action remains reachable through the unified portal. Legacy CGI can stay on a compatible host/container and be accessed through a narrow compatibility route. This preserves functionality without making the Mac's new dashboard a Perl application. A live legacy environment and representative historical runs must be established to validate this route; the checkout alone is not evidence of a working deployment.

For features not yet replaced, show an explicit legacy action/link. Do not display a native button whose backend is incomplete. Imported history stays browsable even when workers or compatibility services are offline; a delegated live operation can require its backend to be online.

Git as primary repository with SVN coexistence

Git is the primary source repository tool for OpenCTI modernization and newly integrated suites. Native enrollment prepares reviewable Git working-tree changes; approved changes follow the project's Git review and merge workflow. Track framework sources, suite adapters, configuration and approved master changes in Git, or retain an immutable reference to an independently versioned upstream source. Large runtime artifacts remain in the native artifact store. Existing SVN suites can continue operating through a repository adapter without requiring immediate conversion.

Choose repository authority per source scope and record it in the suite definition. A Git-managed suite uses its Git repository as the authoritative version history. An external suite still maintained in SVN remains authoritative there until an explicit cutover; OpenCTI pins and snapshots its inputs while its integration metadata can live in Git. Each mutable path has one authority and one controlled publication route. Test execution and source retrieval do not implicitly commit, push or synchronize changes.

Mode	Authoritative repository	Coexistence contract
Native OpenCTI and new suite integrations	Git	Git review/merge controls source, adapter, configuration and approved master changes; SVN is optional
External or legacy SVN suite	Its declared SVN repository	A compatibility adapter retrieves a pinned revision into separate staging and preserves required SVN operations; the Git integration records the upstream identity and snapshot digest
Git source consumed by an SVN-only legacy workflow	Git	If required, a designated publisher exports approved content to a controlled SVN compatibility copy; reconcile content and record the Git commit/SVN revision mapping. Direct writes to that copy do not become a second authoring path
SVN-to-Git migration	SVN before cutover; Git after cutover	Capture history/author mapping and repository metadata, reconcile source content, pause writes for final synchronization, record the cutover and keep an auditable rollback route

Use separate Git and SVN workspaces and materialized input snapshots. Preserve the original SVN helpers for suites that still require them, but route writes by declared repository type; do not run the existing SVN-only `addtest.pl` against a Git-managed suite. A native preview states the target repository and intended changes. Applying source enrollment is separate from publishing it; approved master changes also retain the

baseline-promotion review contract.

The official git-svn documentation describes fetching SVN revisions into Git and publishing changes with `dcommit`. This is an optional bridge for a specifically qualified SVN-backed workflow, not a required part of the native Git path. Its history and merge constraints must be characterized before use. A Git-to-SVN compatibility export may instead publish a reviewed content change from a separate staging checkout. Neither mechanism makes arbitrary independent edits in both repositories automatically equivalent; conflicts require reconciliation before publication.

Every source snapshot records repository type, repository identity/URL, immutable revision and logical source path, plus a materialized content digest. For Git, retain the commit ID and any captured uncommitted patch; a branch name alone is insufficient. For SVN, retain repository UUID, URL/path, revision and any local patch. Capture required properties, externals and empty-directory requirements in the snapshot/dependency manifest, and pin external dependencies rather than resolving mutable references during replay. A bridge/export also records the mapping between originating and published revisions, synchronization direction, result and owner.

An unavailable SVN backend blocks only suites or repository operations that require it. Git-only enrollment, execution and replay remain usable without SVN or `git svn`. Bridge installation, repository conversion and remote publication are separate implementation work; this plan change performs none of them.

Suite adoption and script preservation

Existing OpenCTI suite-adoption scripts and workflows are a compatibility requirement. Keep an identifiable original framework snapshot, including executable modes, symbolic links, wrapper variants and supporting modules. Keep required entry points available through the legacy environment until replacement adapters demonstrate equivalent behavior for supported campaigns. Retiring Perl replaces the implementation after qualification; it does not remove suite enrollment or require every suite to rewrite its build and validation commands.

The independent static audit on 2026-11-11 compared 266 tracked infrastructure entries in `/Users/shinmingliu/opcti` at `14440d0e2e284f5a2c2650d74bc72ff9d8a51312` with framework baseline `8c3d603`. All matched byte-for-byte: 255 regular files, including 191 with shebangs, and 11 symbolic links whose targets resolved locally. The scope covered `Scripts/`, `bin/`, `lib/`, `conf/`, `cti_perf/`, `cti_regression/`, `www/`, `configure.pl`, suite `tmconfig` files, CTI/performance make wrappers and `Perf/bin`. It excluded upstream application sources, dTM Java and general documentation. These counts establish scoped source preservation, not a working deployment, full-package completeness or successful external-suite adoption. Phase 0 now retains the original audit report, per-file inventory and reproduction instructions. The reproduction command verifies immutable Git objects; host-path and link-resolution observations remain historical.

Adoption contracts to retain

Workflow	Existing implementation	Required compatibility
Unit enrollment and selection	TM.pl, generateUnitList.pm, group/unit `tmconfig`	Preserve `Src/` unit identity, parent `CHILDREN` enrollment, direct unit selection, hierarchical option precedence, selectors, exclusions and enumeration
Regression adoption	regression.pl, regressionDriver.pm and discovery/comparison modules	Preserve extension/language mapping, multi-source `.list` and `tmconfig.list`, auxiliary files, per-test settings, negative tests, diagnostic/output masters, qualifiers and result tags
Application adoption	application.pl, applicationDriver.pm, GSL wrapper, HMMER wrapper	Preserve `MAKEWRAPPER`, `compile` and `validate`, compiler/flag/library propagation, setup hooks and run generators; successful validation requires a zero phase status and a present, empty `compare.results`
Externally scripted suites	script-driver.pl, scriptDriver.pm, collection/invocation helpers under `Scripts/utils`	Preserve `SCRIPT_TEST_COLLECTION_HOOK`, `SCRIPT_TEST_INVOKER`, selected test names, `TEST_AUXFILES`, saved environments, per-test workspaces, exit handling, masters and comparison scripts
Performance and specialized suites	Perf configuration, `Makefile.perf`, `APPLICATION_SETUP_HOOKS`, `APPLICATION_RUN_HOOK`, vendor-specific build/link/run/compare helpers	Preserve application-driver routing and generator arguments, suite-specific dependencies and wrapper variants; qualify correctness, flag propagation and hardware controls before treating measurements as comparable
Legacy test addition and delegation	addtest.pl, addtest.cgi, repository helpers, legacyTmUnitDriver.sh	Retain characterized source/master/settings enrollment and legacy access; identify SVN/ClearCase prerequisites and the external `UTM run` delegation separately from native replacements

Preserve executable permissions and relative layouts, including the `driverlib` links under `Scripts/utils`, `Scripts/utils/HPT` and `Scripts/utils/codegen`. locateTools.pm resolves executable drivers under `Scripts/drivers` and ordinary hooks/comparators under `Scripts/utils`. The script collector receives the unit source directory and emits test names; its invoker receives one selected name. An invoker beginning with `.` uses a local path, while other invoker names use hook lookup. Application run generators receive the unit name and output-script filename. These lookup and argument contracts must be represented explicitly in a native adapter or retained compatibility route. See adding tests and architecture for the current layout and invocation details.

Audit gaps and migration work

ID	Observed gap	Required treatment and evidence
SA-G1	`addtest.pl` accepts regression additions only and inserts sources/masters through SVN helpers. No general Git-aware suite importer was found in the audited entry points; application and script-suite adoption use manual layout/configuration.	Make the Git working-tree importer primary, with a reviewable change plan, source/master/settings handling, parent enrollment and rollback. Preserve manual and SVN legacy workflows through repository-specific routing and declared authority. Give application and external script-suite onboarding their own contracts rather than claiming the legacy add-test tool already supports them.
SA-G2	Many entry points name `/usr/local/bin/perl`; that path was absent on the audited Mac. bin/gmake supports Linux only and hardcodes `/usr/bin/gmake`, also absent there. `CTI_global.env` retains configuration placeholders and `configure.pl` edits files in place.	Preserve the unconfigured original and record configured copies separately. Qualify the actual Linux compatibility environment or an explicit portability layer, including interpreter/tool paths and configuration changes. The existing synthetic Docker capture is limited evidence and does not qualify every adoption route.
SA-G3	Perennial helpers contain `/path/to/TESTC++` placeholders; RogueWave helpers reference external `Lang/RogueWave` content; other vendor/SDK helpers and the legacy UTM adapter need external prerequisites.	Inventory required sources, harnesses, tools and access per suite. Mark unavailable dependencies explicitly; retained helper files are not evidence that those suites are runnable. Keep original variants while qualifying supported replacements.
SA-G4	The script driver propagates invoker failure only when `RUN_TESTS_CHECK_EXIT` is true, has no collection-hook time limit and rejects `MULTIPLE_ITERATIONS`. Enumeration invokes the collector.	Characterize both exit-check settings and output comparison, collection failures/hangs, enumeration side effects and unsupported iteration requests. Preserve raw verdict semantics and document reviewed fixes; native timeout or capability handling must not silently convert these cases into passes.
SA-G5	Legacy `-dryrun` is not a pure planner: `addtest.pl` creates temporary directories, can directly write temporary configuration and performs SVN repository lookup before preview-aware check-in. Its help mentions ClearCase while the current addition path uses SVN; separate check-in helpers retain ClearCase operations.	Keep legacy preview isolated and label its behavior. A native preview must disclose exact intended file/enrollment changes without repository mutation, mail or check-in, and applying it must preserve unrelated files and approved masters. Qualify source/master/settings cases in a disposable repository copy.

These are static source findings, not reproduced runtime failures or completed remediation. The modernization backlog must record an owner, prerequisites, evidence and disposition for each `SA-G` item before the affected workflow switches implementations.

Adoption acceptance gate

- SA01: Reconcile original script/configuration hashes, executable modes and link targets against the retained baseline. Record configured copies and intentional changes separately; keep the original compatibility entry points reachable.
- SA02: Enroll and discover a small regression unit, including a multi-source case and expected compile failure. Verify direct/group selection, skips, per-test overrides, masters, result tags and independent failure replay.

- SA03: Adopt an application through its make wrapper. Inspect actual compiler/flag propagation; exercise compile failure, validation failure, missing/nonempty `compare.results` and success. Verify setup/run hook arguments and retained workspaces.
- SA04: Adopt an externally scripted suite through collector/invoker hooks. Exercise file/directory cases, auxiliary files, local invoker resolution, both exit-check settings, output mismatch, collection error/timeout, enumeration side effects and unsupported iterations. Preserve failed invocation and comparison evidence.
- SA05: Characterize legacy regression addition and preview in an isolated configured repository. Qualify native Git source/master/settings addition, group enrollment, conflict handling, preview/apply and rollback; include distinct application and script-suite onboarding fixtures. Do not require a live SVN/ClearCase service for the native workflow.
- SA06: Qualify supported specialized/performance adapters with declared dependencies; unavailable UTM, vendor sources, SDKs or tools produce explicit capability/prerequisite failures. Preserve legacy access until replacement acceptance passes for campaigns that use them.
- SA07: Exercise Git-only enrollment with SVN unavailable, pinned SVN suite retrieval and a declared repository cutover. For each enabled bridge/export, verify revision mapping, content/properties/externals reconciliation, duplicate/retry handling, conflicting edits and publication rollback. Confirm previews identify the write target, a source scope never has two authoritative writers, and legacy SVN helpers cannot write to Git-managed suites. Qualify publication against disposable repositories before connecting live ones.

Assign owners and build these fixtures in phase 0. Carry the applicable scenarios through Docker and remote execution, including result ingestion and retained debugging environments. A workflow can retire its legacy implementation only after its applicable scenarios pass and a rollback path exists; keep unsupported historical material labeled and preserved.

Contracts that permit incremental replacement

Define a small set of versioned records and backend operations. Prefer module/process boundaries over translating Perl source line by line.

Contract	Minimum content
Test definition	Stable ID, family, language/toolchain adapter, source/input references, phases, expected results, comparators, resource needs
Suite adoption plan	Repository type/identity and authority, existing driver/wrapper/hook identities, source and dependency snapshots, configuration/enrollment changes, masters and flag mapping, preview/apply/publication provenance, supported capabilities and rollback
Resolved run plan	Source repository identity, immutable revision, captured local patch and materialized content digest, selected tests, compiler build, resolved options with origin, image digest, target architecture, backend, baseline reference
Attempt	Run/test/attempt identity, backend job identity, current execution and collection states, timestamps, cancellation information
Phase result	Phase identity, command argument vector, environment reference, exit/signal/timeout, stdout/stderr artifacts, expectation and verdict

Contract	Minimum content
Result event	Schema version, attempt ID, monotonically ordered per-attempt event identity, idempotency key, status or artifact manifest
Artifact manifest	Logical path, type, size, digest, provenance and required/optional collection policy
Failure triage environment	Source/harness/baseline snapshots, original and replay scripts, resolved commands/environment, image/toolchain references with retention guarantees, workspace evidence, mount graph, target capabilities and preservation/replay state
Baseline version	Immutable approved oracle/input/artifact references, compatibility scope, review record and predecessor
Known-failure annotation	Matching scope, reason/comment, dates and change history, separate from test outcome and baseline

A language adapter discovers or describes tests and resolves tool-specific phases/comparators. An execution backend submits, polls, cancels, and collects a resolved plan. A result adapter normalizes legacy or native output. The dashboard reads the same local result/query API for all three.

Keep raw imported labels alongside normalized statuses, including unknown legacy tags. Preserve the 48 declared result categories and expected-negative-test semantics. Separate execution outcome from collection completeness: a remotely successful test with missing required evidence is not a fully collected successful local result.

Use structured plans and JSON/JSONL exchange records initially. Tool and schema validation must reject unsupported protocol versions rather than guessing. Retain readable command/environment reproduction exports, but do not depend on serialized shell environment files as the only run specification.

Coexistence and rollback

Introduce replacement selection per campaign/unit capability, for example legacy execution with new reporting, native discovery with a validated legacy execution adapter, or a fully native driver on selected units. Do not mix two drivers on the same mutable workspace. Shadow comparisons use separate scratch directories and the same immutable inputs.

For the first slices, import legacy results without changing the old runner. Characterize the log/parser/cache behavior before replacing option resolution or discovery. Keep legacy `.opt`, `tmconfig`, and `.sched` usable through the compatibility path. New declarative manifests are additive; converting an existing campaign is a reviewed step.

Legacy `.sched` files are executable Perl and known-failure caches can contain Perl-specific serialized data. Use the compatible legacy reader in an isolated, trusted migration path when needed; do not pretend Python can safely infer those values by parsing source text. Capture normalized exports, raw original files, and import provenance. Any narrowly scoped compatibility glue has an owner and retirement gate; new product modules are not written in Perl.

For each mutable operation, nominate exactly one writer during transition. In particular, known-failure edits, baseline promotions, and schedule edits must not be independently authored in both old and new stores. Before a module switches writers, snapshot its state, drain or queue pending edits, reconcile the import, and verify

rollback/replay. A raw cache restore is not sufficient rollback after new edits have occurred.

The new dashboard can show new-language results without forcing them through an old category model. A compatibility exporter can reproduce legacy logs for the subset that maps faithfully, but does not make old CGI understand arbitrary DSL phases. Users retain old features through the portal while new results use the richer native views.

Docker execution with Mac-native evidence

For local Linux testing, use a Linux worker workspace, writable scratch, read-only input/baseline snapshots, and a dedicated host-backed export directory. The native Mac service verifies and ingests exported manifests, logs, result records, and selected binaries/IR. Docker Desktop supports sharing Mac paths with its Linux VM, as described in Docker bind mounts.

Do not base the Linux source tree on the existing case-insensitive host checkout. Materialize the Git revision inside a case-sensitive Linux volume, apply an explicitly captured worktree patch when testing uncommitted changes, and record both identities. Represent Mac-native artifacts by digest with original names in manifests so `X.C` and `x.c` can coexist as distinct objects without renaming test identities. Reconstruct original paths in case-sensitive worker storage when rerunning.

Approved baselines stay on the Mac; a container receives an immutable snapshot or requested artifacts and returns proposed replacements. Remastering becomes a reviewed promotion with a preview, preserving the previous baseline. Keep legacy master-file format usable through an export/materialization adapter where the legacy runner still needs it.

Record native/container OS, CPU architecture, compiler target, image digest and execution mode separately. If an Apple Silicon Mac runs emulated x86 work, label it and do not compare its timings with native x86 performance baselines. Container execution on the Mac does not imply local access to the remote GPU.

Failed-test debugging environments are a core product contract

CTI's retained work environment is its most important value: a developer can inspect the actual failing case, compile it again, execute it, and compare its output with the original baseline. Logs and database rows alone do not satisfy this requirement. Every backend and language adapter must preserve this workflow; dashboard parity includes access to this environment, not merely links to logs.

A failed attempt must have both preserved evidence and a reconstructable writable debugging workspace. The original container need not remain alive, but deleting it must not remove the required state. Preserve the failed workspace before cleanup; do not reconstruct its contents solely by rerunning the test, which could lose the original failure evidence.

Required contents

Retained item	Contract
Test sources and inputs	Exact consumed files, including generated sources, headers, auxiliary files, datasets, uncommitted changes and seeds; stable logical names and digests
Harness dependencies	Unit support objects/libraries, wrappers, hooks, filters and comparator implementation needed by that test; include shared unit dependencies rather than copying only the leaf source
Compile script	A readable executable entry point preserving compiler invocation, flags, environment, working directory, link steps and required inputs
Execute script	A separate readable entry point preserving arguments, runtime libraries, input redirection, limits and device requirements
Compare script	An independently runnable entry point using the original comparator, baseline, normalization and tolerance settings; it must also accept output from a developer's new run
Baseline	The actual version used by the failed attempt, materialized or retained through a durable local object reference; never silently resolve to the latest master
Failure evidence	Original stdout/stderr, diagnostic/output files, phase statuses, generated code/IR, and relevant objects/executables; preserve produced core/debug files where available
Execution environment	Retained image by digest or verifiable image archive, mounted compiler/runtime/dependency snapshots, relevant environment and resource/capability requirements
Mount and path specification	Every required container destination, snapshot identity, access mode, working directory and symlink dependency; separate original host locations from replay locations

A possible materialized debugging view is shown below. Names are proposed interfaces, not files implemented by this documentation change:

```
triage/<run-id>/<test-id>/<attempt-id>/
  manifest.json
  sources/                 # materialized from preserved source objects
  inputs/
  baseline/                # baseline used by this attempt
  harness/
  environment/             # resolved settings, image/toolchain, mount graph
  scripts/
    compile.sh
    run.sh
    compare.sh
    replay.sh              # reconstruct container and invoke a phase
    debug-shell.sh         # reopen a writable workspace interactively
  original-workspace/      # evidence preserved at failure
  logs/
```

The native Mac store can physically hold these files in content-addressed storage and materialize this logical view on demand. It must preserve both case-distinct source names even on a case-insensitive Mac filesystem; reconstruction occurs in case-sensitive Linux storage. An archive/object bundle and a digest manifest are valid

native storage, whereas a path to an ephemeral Docker volume alone is not durable local preservation.

Generated shell entry points may call the new Python phase runner. Their contract is that the developer can read the commands and invoke compile, execute and compare independently without needing a live controller, dashboard or database. During legacy execution, retain CTI's original scripts and `.env` files as well as any new launcher wrappers; do not replace those original scripts with only a normalized JSON record.

Mount-aware preservation and replay

1. Before execution, capture immutable source/input/baseline references, generate the phase scripts, and resolve the required mount graph. Use controlled snapshots for inputs; hashing a mutable host file after a failed run does not establish what the compiler originally read. 2. Mount captured inputs and baseline read-only wherever the adapter permits. Keep mutable build/run state in a per-attempt workspace. If a legacy suite writes into its source tree, supply a private writable copy seeded from the snapshot, and retain both the initial inputs and failed state. 3. Record original container-visible paths. Replay materializes the saved snapshots and mounts them at the same destinations inside a fresh container, even if the original Mac checkout moved. Do not let preserved scripts follow absolute symlinks into today's mutable checkout or master directory. 4. Freeze and collect the failed workspace, scripts, mount manifest and baseline references before container/volume cleanup. Capture required files from container layers, volumes and bind mounts; a container image alone does not contain mounted host files or the failed writable workspace. 5. Verify the local bundle and its dependencies, then mark preservation complete. An image digest alone is insufficient if the referenced image can disappear: retain the image or an accessible verified archive and pin external toolchain artifacts needed for replay. Explicitly record prerequisites that must be reprovisioned, rather than claim a self-contained environment when it is incomplete. 6. Open debugging sessions as new writable copies. Compile, run and compare can overwrite their own session outputs but cannot alter the original failed evidence or approved baseline. Record a new attempt/session relationship when the developer changes a source, option, compiler or baseline.

Use stable container paths for new adapters; legacy adapters may retain their original absolute paths by reconstructing mounts appropriately. Include permissions, executable bits and relevant symlinks. Record needed environment values while replacing credentials with explicit reprovisioning requirements; never silently omit a requirement and call the replay complete.

Shared support harnesses and mounted compilers are dependencies of the debugging environment. Pinning only the test's source revision and image is insufficient when the compiler, runtime, comparator or baseline came from an independently changing mount.

Developer workflow and dashboard behavior

A failed-test page provides access to the preserved sources, original evidence, scripts, baseline and mount/environment details. It offers Open debugging workspace, Recompile, Execute, Compare and Download triage bundle workflows. These actions use the saved attempt configuration by default. Replaying against a newer compiler or baseline is an explicit new session and retains a link to the original failure.

The first session-opening CLI and opt-in dashboard adapter now implement private local Docker workspaces, captured path/environment restoration, interactive shells, reconnect/stop/remove and command execution for supported legacy captures or a versioned archive/mount specification. A registered failed-case click opens a new macOS Terminal window through a native URL handler. Original run-log digests guard against reused log paths, and existing artifact links remain available. Synthetic legacy CGI/browser/Terminal acceptance, real GDB and independent source/work sessions are verified; full dashboard/production/site-specific acceptance remains open. Credential-backed captures require explicit inspection mode until authorized provisioning is integrated.

The debugging workspace supports an interactive shell and, when configured, debugger attachment to a build with suitable symbols and backend capabilities. Interactive debugging is not a promise to checkpoint/restore the exact live process. For nondeterministic failures, preserving the environment and inputs makes investigation possible but does not guarantee the failure recurs.

For a remote GPU failure, the Mac retains the bundle and can inspect sources, logs and IR offline. Execution/debug replay requiring the GPU is submitted to a compatible remote target using the saved environment and device requirements. It must not silently fall back to CPU execution or claim that the Mac reproduces the GPU environment.

Retention and acceptance gate

Track preservation separately from the test verdict: pending, complete, incomplete, or failed, with a missing-dependency list. Historical legacy imports may have dangling source links; show that honestly rather than labeling every old work tree replayable. Cancellation, timeout, worker loss and collection errors must not masquerade as fully preserved failures.

Failure preservation overrides normal successful-test cleanup and container auto-removal. Pin its required dependency objects against garbage collection. A controller/worker should save inputs and scripts before launch and spool failure evidence durably; on disk exhaustion, interruption or host loss, retain what is available, report incompleteness, and retry collection when possible. Remote workers must not remove pending bundles before the Mac acknowledges verified collection. Define an explicit storage-pressure policy instead of silently deleting pinned evidence.

The phase-2 Docker acceptance test must deliberately produce a compile failure, runtime failure and baseline mismatch, preserve each environment, delete the original containers and disposable volumes, move or change the original source checkout and current baseline, and then reconstruct a fresh debugging workspace using only the retained bundle and declared execution prerequisites. Independently recompile, execute where applicable, and compare; verify the original scripts/evidence/baseline remain unchanged. Extend this gate to case-distinct filenames, shared unit harnesses and a toolchain supplied by a mount.

Apply the same test to native adapters, the DSL and remote GPU collection before routing production campaigns to them. This is a mandatory exit gate, not a later convenience feature.

Remote GPU execution and return of results

Use an SSH backend first. It should submit a versioned job bundle to a persistent remote job wrapper/worker, receive a stable remote job ID, and exit without tying the job's lifetime to the SSH connection. Containers are optional per job; GPU requirements belong to the target/executor contract, not to the DSL parser.

The workflow is: resolve locally, transfer inputs, acknowledge remote job, allocate device resources, run, spool events/artifacts remotely, reconnect/poll, copy missing artifacts back, verify digests, ingest locally, and acknowledge collection. Retain remote data until required local collection is acknowledged, subject to an explicit retention policy.

Mac sleep or network loss must not mark a job failed or trigger an automatic duplicate run. Show last-contact time and disconnected/unknown state. Resumption queries the saved job ID and replays events idempotently. Cancellation is a recorded request until the worker confirms the process/container stopped. A rerun creates a new attempt rather than overwriting the previous one.

Record device vendor/model/UUID, architecture, driver/runtime/toolkit, compiler build, image, selected devices, inputs, seed, and measurements. Support exclusive device leases for performance work; testing on a busy shared GPU must not silently produce a comparable performance measurement. Coordinate allocations with all participating jobs on that host; if the host already has a scheduler, integrate with it rather than layering an uncoordinated private lock over it.

For NVIDIA Docker workers, configure the host GPU/runtime integration and explicit device exposure using the NVIDIA Container Toolkit contract. Other GPU vendors need their own capability/runtime adapter; NVIDIA is not assumed to be the DSL's only target.

A remote Docker daemon cannot bind-mount the Mac client's files: bind mounts resolve on the daemon's host. Therefore remote jobs need explicit staging and result transfer, even if Docker is also used remotely. See Docker's remote mount constraint.

The local database and collected artifacts remain available after the remote job/container is removed. Completed history is offline-readable; active monitoring resumes when connectivity returns. Large data and IR retention should be selectable, but required baseline/result evidence must never be silently omitted.

New-language and Open64 DSL testing

Replace extension-to-C/C++/Fortran assumptions with named toolchain adapters. Keep the test family, language, target, and backend independent: a DSL can have regression, application, and performance tests, each executing locally or remotely as appropriate.

A DSL adapter should describe whatever phases the actual compiler needs, such as parse/type checking, lowering, IR validation, optimization, code generation, host link, device execution, and numerical comparison. These are candidate extension points, not assumptions about the DSL's implementation.

Support diagnostic expectations, IR snapshots or structural assertions, executable output checks, numerical tolerances/reference implementations, expected rejection, and measurements. Numerical tolerance, seed/input generation, device capability, and comparator version must be explicit. Do not reduce DSL testing to adding a suffix that launches a compiler and compares text.

The DSL pilot should use representative real cases from the Open64 project: a positive compiler test, an intentional diagnostic failure, a semantic/output test, and a remote GPU case if that backend exists. Phase 0 must establish the real compiler commands, dependencies, language identity, oracle, target requirements, and suitable small fixtures before implementing the adapter. The framework should not require a DSL compiler source change just to submit a test.

Delivery sequence and acceptance gates

The detailed Phase 1-3 implementation plan (Part II), prepared on 2026-11-13 from the latest Phase 0 evidence, defines 19 proposed PR-sized milestones, dependencies, accountable roles, concrete acceptance checks and reversible release gates. Its evidence baseline is PR #17 commit `c6e06a8`; that proposal and formal creator signoff remain pending at this planning revision. The plan supports preparation without implying production cutover approval.

These phases are ordered by usable deliverables, not estimated calendar dates. Dashboard work starts first and remains an acceptance condition in every later phase.

Phase	Deliverable	Required evidence before advancing
0. Establish compatibility evidence	Failure-triage fixtures, dashboard ledger, suite-adoption inventory and SA01-SA07 fixtures, Git/SVN authority map, representative legacy archives/live campaign, schemas, runtime/target inventory, DSL/GPU pilots	Archive can be read with the legacy reader; retained workflows and SA-G1 through SA-G5 have owners, prerequisites and acceptance scenarios; source preservation and repository identities are reconciled and unavailable data/dependencies are listed
1. Native dashboard over legacy results	Mac-native result store/API and useful history/failure/artifact browsing; legacy fallback links/actions	Original failure scripts/workspaces remain accessible, with incomplete dependencies reported; imported counts, categories, historical comparisons, comments and artifacts reconcile with fixtures; history survives container shutdown; existing operational workflows remain reachable
2. Docker execution vertical slice	Controller submits a small legacy CTI run in Linux Docker and retains failure debugging bundles on Mac	Delete original containers, change/move original mounts, reopen retained workspace, and independently recompile/run/compare against the saved baseline; also pass case-sensitive reconstruction, live status and retry-safe collection
3. Incremental native harness	Python phase runner, regression/application/external script-suite adapters, primary Git enrollment with optional SVN adapter and campaign/unit routing	Applicable SA01-SA07 scenarios pass before each adoption route switches; native and legacy runs match on characterized failures, timeouts, missing evidence and option overrides; failed cases pass retained-environment replay; campaign routing, repository authority and enrollment rollback work
4. Native DSL adapter	Real Open64 DSL tests using the same result/dashboard contracts	Diagnostic, IR/semantic and applicable numerical cases preserve sources, phase scripts, comparator/baseline and failed work state for independent replay; no dashboard specialization required to add a language
5. Remote GPU backend	Persistent SSH submission, capability selection, GPU lease, resumable collection	Mac sleep/disconnect, worker failure, duplicate event replay, cancellation and collection retry are exercised; locally stored results and failure triage bundles remain after remote cleanup, with GPU replay verified on a compatible target
6. Complete native parity and retire legacy modules	Native actions/reporting/scheduling, Git-primary suite onboarding with required SVN compatibility and pool controls; remaining harness migration	All feature-ledger and applicable SA01-SA07 scenarios pass; every SA-G gap has a recorded disposition; historical sources/data remain available; mutations migrate with rollback; supported native campaigns run without Perl or legacy CGI/dTM dependencies, and any optional Perl-based bridge remains isolated with a declared dependency

Schema design in phase 0 must include remote attempt identity and DSL phases so later backends do not force a reporting rewrite. A minimal remote submission experiment can happen early after the contracts stabilize, but dashboard parity and collection durability remain gates. Performance acceptance on real hardware follows successful remote correctness and lease handling.

Phase 0 working artifacts

The phase 0 review package now consolidates the locally executable reference work, ten proposed contracts, adoption inputs, repository authority and runtime inventory, discrepancy register and one-command audit. The maintainer reports no historical campaign archive and designates the GitHub repository creator as peer reviewer. Technical preparation is complete for this declared reference scope; creator signoff remains pending. Missing historical/GPU/vendor coverage stays explicit, and no production cutover or feature retirement is approved by this status.

The dashboard feature preservation ledger records feature IDs, implementation evidence, owner roles, fixture specifications and dashboard scenarios. The failure triage acceptance scenarios define legacy characterization and preservation/replay gates for Docker, mounted dependencies, standalone debugging, the DSL and remote GPUs.

These documents start phase 0; they do not establish a passing reference deployment. A synthetic legacy Docker capture now records real FT01 through FT03 failures and independent phase replay. An FT04 follow-up verifies archive-only reopening after disposable-state deletion and source/baseline relocation for the same synthetic subset. An FT05 isolated promotion experiment now verifies preserved-baseline replay and records the legacy mutable-master hazard for synthetic FT03. An FT06 mounted-dependency experiment verifies pre-run content snapshots and replay after same-path compiler/runtime/input/comparator and dirty-header mutations in a new synthetic case. An FT07 shared-support experiment now verifies captured C support objects, headers, filters and link closure after deleting original sandbox work/dependency paths, with a separate explicit rebuild. An FT08 writable-application experiment now preserves initial/generated failure state and verifies overlapping private source/work sessions with mutations only in A. An FT09 platform/debugger experiment now verifies native Linux ARM64 phase replay and real GDB attachment without added capabilities; incompatible foreign-worker records are explicitly descriptor tests. An FT10 collection fixture now verifies retry-safe transfer, idempotent failure events, corruption/storage recovery, acknowledgement/pin guards and incomplete lost-spool reporting without rerunning tests. An FT11 result-semantics fixture now characterizes expected failures, partial/unit loss, cancellation precedence, signal/process status and unknown imports, with a negative-runtime false pass explicitly preserved as a legacy discrepancy. An FT12 rerun/reuse fixture now verifies original pinned evidence and replay after isolated reruns, private edits and destructive cleanup; legacy missing-header settings and nested-history loss remain explicit gaps, with new native workspace guards tested separately. An FT13 prerequisite fixture now verifies actual isolated image loss/offline reload, missing or incompatible runtime state, incomplete dangling imports and explicit recovery, with credential references retained without values. An FT14 isolated lifecycle subset now verifies persistent jobs, disconnected/resumed collection, acknowledgement/cleanup gates, fixture lease fencing and CPU-control replay after worker deletion. No compatible remote GPU worker is available; full FT14, actual Mac sleep, SSH/WAN transfer and physical GPU leases remain unqualified. An FT15 application/script characterization now verifies original make/check/marker oracles, external glue and file/directory inputs, and independent saved-phase replay after container deletion. Its stale-empty-marker run-hook false pass is retained as a reference discrepancy without changing the framework. An FT16 interruption experiment now verifies actual stock timeout and isolated before/after-start cancellation, forced descendant termination, paused-worker reconnect and stored-workspace recovery without duplicate dispatch. It retains partial streams, keeps worker-loss preservation explicitly incomplete, and verifies Docker resource cleanup plus logical lease fencing. DA15/live dTM, remote transports and physical GPU/performance reservation acceptance remain open. An FT17 filename/permission/link experiment now verifies native manifest/object/archive storage and fresh case-sensitive Linux reconstruction of C/C++ case pairs, modes and links. The case-insensitive Mac extraction control blocks before writing. A separate legacy same-basename work overwrite is preserved as a reference discrepancy, requiring full-identity attempt isolation in a future adapter. An FT18 isolated access experiment now verifies actual browser evidence inspection, native Terminal handoff and standalone interactive replay from a relocated portable kit after dashboard shutdown. It retains an observed in-page-anchor rewrite defect and the missing product bundle/mount/completeness views; full FT18/DF38 remains

open. The global URL handler is unchanged and remote GPU access is unqualified. An FT19 real CPU DSL experiment now preserves a retained Open64 LNO compiler crash, source-interleaved native IR and pinned compiler/library/input/oracle closure through worker deletion and archive relocation. The original crash remains distinct from fourteen passing semantic controls in a separately labeled portable CPU lane. Historical IPL recovery, frontend regeneration, generated-program/core or numerical-miscompile variants and production DSL integration remain open. Broader historical campaigns are unavailable per the maintainer. The new measured legacy reader/CGI reference covers completed five-test histories and limited filters/catalogs, with a negative incomplete-count discrepancy retained. Technical preparation ownership and creator review are recorded in the closeout catalog; organization operations ownership, remaining runtime scenarios and replacement acceptance remain open. The peer-review PDF refreshed on 2026-11-13 includes the latest Phase 0 status and detailed Phase 1-3 implementation plan. The Markdown and generated HTML remain the maintained sources; regenerate the PDF after subsequent plan changes.

The peer-review PDF refreshed on 2026-11-13 contains this updated roadmap and the complete detailed Phase 1-3 implementation plan as a second part. The original November 11 PDF is an earlier snapshot; maintained Markdown and generated HTML carry subsequent changes and require a matching PDF rebuild.

First implementation scope

Start with the evidence ledger, suite-adoption inventory, import contracts, and one dashboard-to-result vertical slice. Use a small legacy regression run and one application result, including a known failure, a new failure, and accessible logs/masters; establish an external script-suite fixture alongside them. Make that history native and durable on the Mac, with a working link to any legacy action not yet replaced.

Then demonstrate a real local Docker run whose output lands in the same dashboard without manual copying and whose failed environment can be reopened after container deletion and original mount changes. That validates the central architecture before replacing compiler-driver logic or expanding scheduling. No existing suite metadata or baseline should be rewritten as a prerequisite.

The completion criterion for the whole transition is concrete: failed tests retain usable source/compile/run/compare debugging environments independently of the original containers and mutable mounts; suite enrollment and regression/application/script-suite adoption pass their acceptance gates with original scripts preserved; the Mac dashboard retains the original functional workflows, old and new results coexist, the DSL is supported through adapters, Docker and remote GPU runs collect reliable local evidence, and active campaigns no longer need Perl. Remove compatibility runtimes only when those conditions are demonstrated for the supported campaigns.

Part II - Phase 1-3 implementation plan

Decision and evidence baseline

Peer-review revision: November 13, 2026. This plan specifies 19 proposed implementation pull requests after a preparation decision, G00. It translates the modernization roadmap into reviewable changes, owners, dependencies and executable acceptance objectives. Milestone IDs are planning identifiers, not existing GitHub pull request numbers. None of the new product APIs, commands or acceptance results described below is claimed to exist already.

The evidence baseline is commit `c6e06a80c7ddb75587ec91d2d3c7b971250b9f7f`, proposed in PR #17. Upstream `master` was `c452c0a45538910a28a73be8c8824a072499bd5d` when this plan was prepared. PR #17 was open, the technical Phase 0 package was ready for peer review, and creator signoff remained pending. Merge and peer signoff are separate decisions. Reconfirm both before authorizing a production route or campaign cutover.

The retained closeout record records 29 successful commands, including 185 Python tests, JavaScript checks and 43 generated documentation files at that baseline. Seven additional auditor tests reject fabricated approval/coverage claims. Those results characterize reference tools and fixtures; they do not qualify the native components proposed here. Adding this guide changes the generated-document count; retain the original record unchanged.

Evidence that drives implementation:

Input	Established reference and implementation consequence
Measured reader/CGI corpus	Two synthetic campaigns, each with five stable test IDs and previous/current histories. Previous: 5 tests, 3 passes, 2 failures. Current: 5, 2, 3, including 1 known and 2 unknown failures. Aggregate current failures: 6/10; previous: 4/10. A derived partial log exposes negative unknown counts.
Discrepancy register	Ten observed gaps, P0-D01 through P0-D10, remain traceable. Replacement behavior needs explicit tests and reviewed dispositions; preserve raw legacy observations.
Versioned design contracts	Ten proposed JSON Schema records with 24 structural/cross-field tests. Adopt/version these records deliberately; build the transport, persistence and semantic checks around them.
Adoption fixtures	SA01-SA07 have specifications and some observed legacy subsets. Native enrollment, full group/override discovery and live SVN/bridge publication remain acceptance work.
Runtime inventory	Native Linux ARM64 reference workers and an emulated AMD64 Open64 CPU lane exist. Historical campaigns, a compatible remote GPU worker, organization services and vendor/UTM environments are unavailable or unqualified.
Session CLI acceptance and dashboard access acceptance	Existing restricted sessions and browser-to-Terminal handoff are reusable assets. Preserve `cti-session/v1`, private workspace isolation and verified identity bindings. Product bundle/mount/completeness views still need implementation.
Coverage catalog	Preserve all 38 feature IDs and 16 dashboard scenarios. A completed subset does not retire an entire legacy workflow.

Ownership and working rules

Each milestone has an accountable implementation role and a required reviewer. Named human assignments are pending; the table assigns responsibility without inventing people. Codex can prepare code, fixtures and review artifacts under maintainer direction. That does not appoint an operational service owner or grant peer approval.

Role	Accountable responsibility	Assignment and review
M - Maintainer	Priorities, named owners, compatibility scope, release and campaign-route decisions	Current project maintainer; record the GitHub handle at G00
R - Results/API lead	Record model, ingestion, SQLite migrations, counts, history and query contracts	Named implementer pending; Codex may prepare changes; review by M and T
D - Dashboard lead	TypeScript views, navigation, artifact inspection and accessible action states	Named implementer pending; Codex may prepare changes; review by R and T
H - Harness lead	Plan resolution, Docker worker, native drivers, adapter compatibility and replay	Named implementer pending; Codex may prepare changes; review by O and T
O - Operations lead	Runtime profiles, dependency retention, cleanup, local installation and recovery	Named human operational owner required before ongoing service ownership; review by M and H
T - Test/evidence lead	Fixture identity, negative tests, evidence auditing and discrepancy reconciliation	Named implementer pending; Codex may prepare checks; review by the affected component owner
P - Designated peer reviewer	Phase-level evidence/design review and recorded signoff	GitHub repository creator, as designated by maintainer; account mapping still unresolved

An implementation PR changes one boundary, migration or adapter. Its description identifies its milestone, fixtures, exact commands/results, original-evidence before/after hashes, compatibility impact and rollback. Split further when a schema change, controller behavior and independent UI feature would otherwise require separate reviews. Preserve original framework sources, approved masters and existing receipts. Use new outputs for each validation attempt.

G00 records M/R/D/H/O/T/P assignments, the accepted reference commit, schema-version decisions, the declared CPU campaign subset and how legacy actions remain available. Offline design and fixture preparation can proceed while signoff is pending; default routes stay legacy and no production writer changes until the relevant approval is recorded. An unavailable organization endpoint appears as unavailable with a cause, rather than as a successful fallback action.

Product boundaries and proposed interfaces

Use a Python native service with a TypeScript browser UI, one local SQLite writer and a native content-addressed artifact store. Keep execution in case-sensitive Linux storage. A language adapter resolves phases; a backend dispatches, monitors, cancels and collects them. Workers never update approved baselines or write directly to the Mac database.

Proposed code boundaries are `cti/contracts`, `cti/store`, `cti/ingest`, `cti/api`, `cti/backends`, `cti/adapters`, `cti/adoption` and `web/`. These are target modules, not existing directories. Review the framework/dependency selection, pinned versions and test setup in P1-01; this plan does not require a particular Python HTTP or UI framework.

Proposed interface	First milestone and contract
`GET /v1/campaigns`, `/runs`, `/attempts/{id}`, `/attempts/{id}/phases`	P1-03. Stable opaque identities, bounded pagination, filter/sort rules, raw plus normalized statuses and compatibility origin.
`GET /v1/attempts/{id}/artifacts`, `/preservation`, `/baseline`; `GET /v1/artifacts/{digest}`	P1-05. Logical names, digest verification, byte ranges where needed, explicit missing dependencies and original baseline identity. A URL never supplies an arbitrary server filesystem path.
`POST /v1/run-plans`, `/run-plans/{id}/submissions`; `GET /v1/jobs/{id}`	P2-01/P2-03. Validated immutable plan, stable job identity and idempotent submission. Register the complete request/response/error schemas before enabling writes.
`POST /v1/jobs/{id}/cancel`; `POST /v1/attempts/{id}/sessions`	P2-03/P2-05. Cancellation is a request until confirmed. Session creation uses authenticated local intent, pinned evidence and capability checks, then reuses the session runtime contract.
`POST /v1/adoption-plans`, `/adoption-plans/{id}/apply`	P3-06. Reviewed path/content preconditions, exact diff and conflict reporting. Applying enrollment and publishing Git/SVN changes are separate operations.
Proposed CLI commands `cti import`, `cti serve`, `cti submit`, `cti inspect`, `cti adopt preview/apply`	Add alongside the matching milestone. Preserve existing `tools/cti-session` and dashboard identity bindings; version any new interoperability layer.

Store campaign/run/test/attempt identities separately. An attempted rerun always gets a new attempt and workspace. Preserve raw legacy tags, source logs and reader exports alongside normalized values; unknown tags stay unmapped. Store known-failure annotations separately from verdicts and baselines. Keep original diagnostic signal, observed process exit/signal and collection status distinct, including the FT19 compiler-crash case. Content installation precedes an acknowledged manifest/index transaction; partial installs are recoverable and never appear fully collected.

Native APIs bind loopback by default and serve only declared artifact objects. Read-only browsing must not start a worker. Mutating actions require deliberate local authorization, origin/CSRF handling and confirmation appropriate to their effect; a visited link never executes a phase or promotes a master. Shared-user, organization authentication and remote deployment remain separately qualified configurations.

Milestone dependency map

The critical path is G00 -> P1-01 -> P1-02 -> P1-03 -> P1-04/P1-05 -> P1-06 -> P2-01 through P2-06 -> P3-01 -> P3-02 -> P3-03/P3-04/P3-05 -> P3-06 -> P3-07. Parallel work uses disjoint component scopes and a fixed shared contract.

Milestone	Deliverable	Depends on	Owner / reviewer
P1-01	Contract and native service skeleton	G00	R / M,T
P1-02	Durable local store and verified artifact installation	P1-01	R / O,T
P1-03	Read-only legacy import and query API	P1-02	R / H,T
P1-04	Native history/failure dashboard	P1-03	D / R,T
P1-05	Evidence, completeness and debugging access	P1-03	D / H,O,T
P1-06	Phase 1 parity subset and operational handoff	P1-04,P1-05	T / M,P
P2-01	Docker backend and submission contract	P1-06	H / O,T
P2-02	Immutable execution input/dependency snapshots	P2-01	H / O,T
P2-03	Durable local dispatch, status and cancellation	P2-02	H / R,O,T
P2-04	Retry-safe collection and retention	P2-03	R / H,O,T
P2-05	Portable debugging and session integration	P2-04,P1-05	H / D,O,T
P2-06	Destructive recovery and Phase 2 acceptance	P2-05	T / M,P
P3-01	Native discovery and option resolution	P2-06	H / R,T
P3-02	Native phase execution and verdict engine	P3-01	H / R,T
P3-03	Regression adapter	P3-02	H / T
P3-04	Application wrapper adapter	P3-02	H / T
P3-05	External script-suite adapter	P3-02	H / T
P3-06	Git-primary adoption transactions and SVN boundary	P3-03,P3-04,P3-05	H / M,T
P3-07	Campaign routing, shadow comparison and rollback	P3-06	M / H,O,T,P

Phase 1 - native results and evidence dashboard

P1-01 - freeze contracts and bootstrap the native service

Owner R; reviewers M and T; dependency G00. Scope: package layout, local configuration, dependency lock files, API version/error envelope, fixture loader and initial CI command. Review each proposed Phase 0 schema against real records and record accepted versions/migrations. Preserve the separate `cti-session/v1` format behind an explicit adapter.

Acceptance: run the existing contract suite; reject unknown versions and conflicting duplicate events; validate fixtures without network resolution. A local health/version endpoint starts without Docker, Perl or organization services, and exits cleanly. Record configuration outside the Git checkout; no private state enters logs or test artifacts. Rollback: stop the new service and retain legacy-only entry points; no existing data migration has occurred.

P1-02 - implement SQLite persistence and artifact transactions

Owner R; reviewers O and T; dependency P1-01. Scope: migrations, campaign/run/test/attempt/phase tables, immutable baseline references, raw status/annotation storage, event uniqueness, artifact objects and a manifest index. Keep source logical names case-sensitive even when the host filesystem is not. Create explicit pending/verified/incomplete installation states and retention pins.

Acceptance: kill the installer before object rename, before database commit and after commit but before acknowledgement; restart and reconcile each state without inventing a complete result. Import the same event twice and assert one stored identity; conflicting content fails. Retain `Case.c` and `case.c` as distinct logical objects. Backup/restore into a fresh state root preserves counts and digests. Rollback: export and verify the native state before restoring a compatible database backup; never point an older binary at a newer schema without a tested migration decision.

P1-03 - import legacy evidence and expose bounded queries

Owner R; reviewers H and T; dependency P1-02. Scope: import the measured reader exports and retained manifests, with provenance and collection completeness. Use the isolated trusted legacy reader only when executable Perl caches require interpretation; the native importer consumes its normalized export and never evaluates Perl. Add campaign/run/attempt/phase queries with stable pagination and documented filter/sort semantics.

Acceptance: import both alpha/beta histories twice with identical totals. Per campaign, previous is 5/3/2 and current is 5/2/3; current has known B, unchanged unknown C and new failure D; expected-rejection E passes. Aggregate failures remain 6/10 versus 4/10. Skip the intervening unfinished history for completed-run comparison. Preserve the raw `[-1 / 1 / 0 / 0]` observation while exposing an explicit incomplete/unreconciled normalized state, never a fabricated zero-failure pass. Include FT11 unknown tags, missing units, cancelled attempts and process/verdict disagreements. Rollback: withdraw the new import namespace; keep raw files and prior imported versions.

P1-04 - build read-only campaign and failure browsing

Owner D; reviewers R and T; dependency P1-03. Scope: campaign/date/options matrix, previous/current comparison, known/new/unchanged categories, filters, sorting, limits, compiler/context details and original log navigation. Legacy actions stay clearly labeled and available only when their configured endpoint is qualified.

Acceptance: browser tests reproduce the measured completed-history subset and verify filter include/exclude, selected campaign, stable pagination and an empty result. Partial/unknown states remain visible. Capture actual screenshots/click transcripts, not only generated HTML. Same-page anchors remain anchors; only explicitly bound launch actions can request Terminal access, closing P0-D06. Original CTI links remain navigable. For a separately labeled generated 10,000-attempt UI fixture, record host/browser versions and at least 20 timed navigations; the proposed local target is p95 <= 2 seconds to the first useful result page. A failed target triggers pagination/profile work, not a claim about measured production scale. Rollback: route views to legacy while native history remains retained.

P1-05 - expose failure preservation and existing session access

Owner D; reviewers H, O and T; dependency P1-03. Scope: attempt detail with original compile/run/compare scripts, source/actual/expected views, original baseline identity, dependency/mount manifest, preservation state and a downloadable inspected bundle. Add byte-range/text limits and safe archive/object access. Reuse the existing session and identity-binding implementation; qualify its source-hash changes with fresh acceptance records.

Acceptance: actual browser navigation opens each FT01-FT03 failure, an FT18 bundle and the FT19 crash with raw diagnostic/process status separate. Missing runtime/IPL/baseline data yields an explicit inspection-only or blocked replay state with named prerequisites. A fragment link cannot open Terminal. A valid deliberate session action opens the declared test/cwd/options; a stale binding is rejected. Service restart and dashboard shutdown do not remove evidence or existing standalone replay capability. Downloaded manifests match digests and disclose excluded runtime dependencies. This closes the selected P0-D07 visibility subset. Rollback: disable new launch bindings and retain legacy access plus exported evidence.

P1-06 - qualify the Phase 1 slice and hand it over

Owner T; reviewers M and P; dependencies P1-04 and P1-05. Scope: one reproducible Phase 1 acceptance command, schema migration/backup rehearsal, browser artifacts, a capability table and per-feature coverage updates. Name the ongoing local service/storage owner and document installation, state locations, limits and recovery.

Acceptance: run the importer/store/API/browser suites from a fresh state root; restart the service, remove original fixture containers and browse retained history without reimport. Reconcile the declared DA01-DA06/FT11/FT18 subsets; leave broad history, annotation mutations and unavailable organization actions explicitly open. Demonstrate P0-D03 attempt-history isolation, P0-D06 anchor behavior and P0-D08 partial-count handling. Require recorded reviewer approval for the declared view subset, not blanket DA acceptance. Phase 1 passes only with durable local history, inspection and a functioning or explicitly unavailable legacy route for every exposed action.

Phase 2 - Docker execution and recoverable debugging

P2-01 - introduce a bounded local Docker backend

Owner H; reviewers O and T; dependency P1-06. Scope: submit/poll/cancel/collect backend operations, validated run-plan API and worker capability requirements. Wrap the legacy runner without changing its drivers. Select explicit OS/architecture/image identity, CPU/memory/time/resource limits and allowed mounts. Preflight Docker/image/dependency availability before creating an attempt that can execute.

Acceptance: submit the small regression control through the controller and observe it through the same native API/dashboard. Missing image, wrong architecture and unavailable interpreter produce prerequisite errors without fallback. Planning/browsing remains usable with Docker stopped. Use the FT09 and FT13 controls and preserve native/emulated execution labels. Rollback: disable submissions for the new backend; existing evidence and legacy direct execution remain available.

P2-02 - snapshot sources, baselines and mounted dependencies

Owner H; reviewers O and T; dependency P2-01. Scope: immutable source/configuration/dirty-patch inputs, initial build state, baselines, toolchain/runtime/comparator closure and a mount graph. Materialize execution in a case-sensitive Linux workspace using full test identity. Pin image/runtime content; represent unavailable objects as prerequisites, not replay-ready references.

Acceptance: reproduce FT04-FT08/FT17 controls. Change source, master, compiler, runtime, dirty header and comparator at their original locations after capture; the attempt still resolves its recorded versions. Two overlapping sessions share immutable inputs but not writable work. C/C++ case pairs retain distinct identities; a host extraction collision blocks before writing. Reviewed promotion creates a new baseline version, preserving the failed attempt's original oracle, closing P0-D01/P0-D05 for this backend. Rollback: materialize from an earlier immutable input record as a new attempt; never rewind an existing attempt in place.

P2-03 - make dispatch and cancellation durable

Owner H; reviewers R, O and T; dependency P2-02. Scope: persist a job/attempt before dispatch, label owned resources, use a unique submission key and reconcile controller restart against worker identity. Record pending/running/disconnected/completed/cancel-requested/confirmed states separately from phase outcomes. Bound process trees and preserve partial streams.

Acceptance: repeat one submission, restart the controller before and after worker creation, disconnect status polling and reconnect without duplicate execution. Exercise cancellation before start and during a child process, stock timeout and forced worker loss using FT12/FT14/FT16 controls. Report cancellation only after confirmation; report unresolved/lost work honestly. Remove only resources owned by this attempt. Rollback: stop new dispatch, reconcile every live job, drain or explicitly cancel it and preserve its spool before returning routing to legacy.

P2-04 - collect verified artifacts with retry and retention rules

Owner R; reviewers H, O and T; dependency P2-03. Scope: staged transfer, content/manifests validation, resumable local collection, durable acknowledgement, object pins and cleanup policy. Keep execution outcome and collection state orthogonal. A successful process with missing required evidence remains incomplete.

Acceptance: FT10-style duplicate events/transfers produce one installed attempt; corrupt/truncated bytes are rejected; disk exhaustion preserves available partial evidence and a useful cause; restart resumes collection without rerunning the test. Neither success nor failure cleanup removes pending data before verified acknowledgement. Baseline, original scripts and required dependency objects stay pinned through later cleanup. Add a restore rehearsal from native object store plus metadata backup. Rollback: disable garbage collection and retain the verified spool/object inventory until reconciliation finishes.

P2-05 - integrate portable debugging sessions

Owner H; reviewers D, O and T; dependencies P2-04 and P1-05. Scope: export a portable manifest/bundle, reconstruct a private session, expose independent compile/run/compare entry points and reconnect supported sessions. Preserve `cti-session/v1` compatibility or supply a reviewed explicit conversion. Edited-session export needs its own versioned derived artifact, leaving original evidence immutable.

Acceptance: open from the new dashboard and from a standalone command after stopping the controller/dashboard. Inspect source, execute each applicable phase and compare new developer output with the original baseline. Show dependency mounts and inspection/replay readiness. Removing a private container and reconnecting preserves declared private volumes; removing a session only removes its owned resources. FT18 anchor/identity guards and original-evidence hashes remain valid. Rollback: withdraw the converter/new button while keeping original session specs and standalone recipes.

P2-06 - execute the destructive Phase 2 recovery gate

Owner T; reviewers M and P; dependency P2-05. Scope: a disposable end-to-end acceptance environment and recorded deletion/relocation transcript. Combine failures from FT01-FT08 with interruption, filename and prerequisite controls; use original sources and retained toolchain snapshots rather than regenerating the failure evidence.

Acceptance: delete original containers and disposable execution volumes, move source/master/toolchain backing locations, stop the controller and remove its temporary query index. Reconstruct from the retained native artifacts and declared retained runtime; compile, run and compare independently where applicable. Verify original evidence/baseline unchanged, case-distinct and shared-harness inputs complete, failed versus incomplete state accurate, and no original mount needed. Rebuild the index and recover the same identities/counts. Record which local CPU subset passed and its prerequisites. Remote SSH, real Mac sleep, physical GPU leases and the broader FT14 gate stay unqualified until their actual targets exist.

Phase 3 - native harness and Git-primary adoption

P3-01 - resolve native catalogs and effective options

Owner H; reviewers R and T; dependency P2-06. Scope: source adapters for `.opt`, group/unit/test `tmconfig`, extension mapping, parent `CHILDREN`, direct selections, skips, auxiliary files and multi-source lists. Preserve option origins and pin the resolved result. Execute trusted legacy option/cache logic through the isolated compatibility reader where needed; avoid pretending general shell/Perl configuration is declarative data.

Acceptance: compare enumerated IDs and resolved options with the same legacy fixture snapshot, including the prepared two-source case, per-test override, exclusion and direct/group enrollment. Component files are not extra tests. Saved rerun settings reproduce the same options; a rerun creates a new attempt, closing P0-D03's resolver

subset. Unsupported configuration blocks with a named reason. Rollback: keep that unit routed to the legacy resolver until characterized.

P3-02 - implement native phase execution and verdict semantics

Owner H; reviewers R and T; dependency P3-01. Scope: phase dependencies, argument vectors, environment/cwd, exit/signal/timeout capture, expected rejection, comparator invocation and structured events using the proven backend/store. Emit readable reproduction scripts and retain originals separately. Resolve verdicts from both expectations and observed outcomes.

Acceptance: use FT11/FT16 controls for ordinary failure, expected compile/runtime rejection, absent unit/test results, unknown tags, timeout/cancellation and interrupted collection. The negative-runtime legacy false pass becomes a documented corrected native outcome with raw legacy evidence retained, closing P0-D02 for the declared case. A zero launcher status never overrides failing tests. The FT19 diagnostic SIGSEGV and observed process signal remain separate facts on import; general native DSL compilation is Phase 4. Rollback: switch the unit's execution route to legacy as a new attempt; preserve native history.

P3-03 - implement the regression adapter

Owner H; reviewer T; dependency P3-02. Scope: compile/link/run/compare recipes, negative compile handling, source grouping, support dependencies, master qualifiers, output padding and comparator/filter rules. Keep compiler and linker flags explicit at their actual invocation sites.

Acceptance: execute SA02 across direct/group selection, multi-source, override/skip, compile-only/link-disabled, negative compile and runtime mismatch controls. Match characterized source/flag/master behavior and document intentional verdict corrections. Reopen an adapter-produced failure after input relocation through the Phase 2 retained environment. Same-basename C/C++ attempts never overwrite one another. Initially qualify the small supported C/C++ subset; Fortran, simulator and vendor environments need their own fixtures before routing.

P3-04 - implement the application wrapper adapter

Owner H; reviewer T; dependency P3-02. Scope: preserve `MAKEWRAPPER`, actual compiler/flag/library/`UNIT_MAKEVARS` propagation, setup hooks, run-generator arguments and `compile`/`validate` behavior. Keep existing wrappers as interfaces instead of translating upstream build systems.

Acceptance: SA03/FT15 controls cover compile failure, nonzero validation, missing/nonempty/fresh-empty comparison markers and a custom generator. A stale empty marker plus failed run cannot pass, closing P0-D04. Preserve raw marker files and the actual phase status; record a fresh marker's relationship to the attempt rather than relying only on file existence. Verify initial/generated state and independent replay with the FT08 fixture. Rollback: keep a nonqualified application on the legacy driver with its original wrapper intact.

P3-05 - implement the external script-suite adapter

Owner H; reviewer T; dependency P3-02. Scope: bounded collection, stable file/directory test identities, auxiliary inputs, invoker lookup, saved environments, output comparison and explicit iteration capabilities. Preserve characterized legacy stdout-then-stderr concatenation when comparing a compatibility oracle.

Acceptance: SA04/FT15 exercises successful/empty/failed/hanging collectors, enumeration side effects, local `./` invokers, file and directory cases and exit 7 with both exit-check settings. A collector timeout is an explicit collection

failure. Unsupported iterations are rejected before execution; enabling new iteration semantics requires a separate fixture/version decision. Mark disabled exit checking visibly and preserve actual exit status even when a compatibility verdict follows output. Close P0-D10 for the supported route. Rollback: retain legacy hooks and the original lookup layout.

P3-06 - add reviewed Git adoption transactions and SVN routing

Owner H; reviewers M and T; dependencies P3-03, P3-04 and P3-05. Scope: native preview/apply plans for regression, application and script suites; source/master/settings/parent-enrollment diffs; content preconditions; conflict detection; exact-path rollback; and declared repository authority. Use a disposable Git worktree/staging area, preserving unrelated paths and original failure evidence. Keep applying enrollment separate from commit/push and approved-baseline promotion.

Acceptance: SA01/SA05/SA07 works with SVN absent for Git-owned scopes. Preview performs no test execution, mail, check-in or baseline mutation. Modify a precondition after preview and require a new review; replaying an applied transaction is idempotent. Roll back only paths owned by that transaction and refuse to overwrite intervening edits; the unrelated sentinel survives. Reject legacy SVN writers on Git-owned paths. Test pinned SVN metadata/properties/externals and explicit unavailable-backend behavior with synthetic fixtures. Activate a real SVN reader or publication bridge only after a separately configured repository, owner and isolated integration test pass. SA-G1/SA-G5 receive native treatments; live SVN and vendor availability remain conditional.

P3-07 - qualify campaign routing and reversible cutover

Owner M; reviewers H, O, T and P; dependency P3-06. Scope: explicit campaign/unit capability routing, separate shadow workspaces, comparison reports, release records and a reversible default-route change for a small named CPU campaign. Maintain one writer for enrollment, annotations, baseline promotion and repository publication in each scope.

Acceptance: legacy and native attempts use the same immutable sources, resolved options, baseline and compatible worker; compare test identities, commands/flags, raw/normalized outcomes, artifacts and preservation state. Catalogue every difference as a bug, expected correction or blocked prerequisite. Qualify SA01-SA07 only for their declared scope; demonstrate SA06 missing vendor/SDK/UTM prerequisites without a false pass or fabricated performance result. Revert routing and recover history/private sessions without rewriting evidence. Native routes remain opt-in until approvals are recorded. Whole-framework Perl retirement, broad dashboard mutation parity, DSL execution and remote GPU production rollout retain later phase gates.

Discrepancy and coverage commitments

Phase 0 item	Required milestone and acceptance evidence
P0-D01 mutable masters	P2-02 immutable baseline; P3-06 reviewed promotion/enrollment boundary
P0-D02 negative-runtime false pass	P3-02 explicit expectation/outcome truth table with preserved legacy result
P0-D03 rerun options/history loss	P1-02/P1-06 separate attempts; P3-01 pinned effective options; P3-07 routing replay
P0-D04 stale application marker	P3-04 status plus attempt-scoped comparison-marker validation
P0-D05 same-basename overwrite	P2-02 full identity/case-sensitive workspace; P3-03 regression control
P0-D06 anchor rewriting	P1-04/P1-05 real browser fragment versus deliberate launch tests
P0-D07 missing product preservation views	P1-05 inspect/mount/completeness/bundle views; P2-05 portable replay
P0-D08 negative partial counts	P1-03/P1-04 raw observation plus explicit incomplete normalized state
P0-D09 reconstructed IPL provenance	P1-05 imports/displays the limitation; exact original DSL closure stays a Phase 4 requirement
P0-D10 collector/iteration limitations	P3-05 bounded collection and explicit capability errors
SA-G1 Git importer	P3-06 preview/apply/conflicts/rollback for all three adoption routes
SA-G2 deployment portability	P1-01 configuration; P2-01 preflight; P3-07 per-campaign qualification
SA-G3 external prerequisites	P2-02 dependency manifest; P3-07/SA06 explicit unavailable state and retained compatibility access
SA-G4 script semantics	P3-05 collection/exit/oracle/iteration controls
SA-G5 preview side effects	P3-06 isolated transaction with no execution/mail/publication on preview

DA01-DA06 receive only the measured browsing/navigation subsets in Phase 1. DA07-DA16, including broad annotation mutations, cache/admin actions, rerun/remaster/enrollment UI, live dTM/pool controls and operational behavior, remain tracked individually in the original ledger. Phase 2/3 may supply backend evidence for selected scenarios, but neither phase silently marks all of them passed. Keep the other legacy actions reachable when their actual compatibility deployment is available, or expose an explicit unavailable state. Full mutation/admin parity remains a later release obligation.

FT01-FT13 and FT15-FT18 supply local reference controls for these phases. FT14 contributes the measured isolated lifecycle subset, with WAN/Mac-sleep/GPU behavior still open. FT19 supplies retained CPU crash/import/provenance controls; its separate semantic lane is not evidence that the original crash was fixed or that production DSL compilation is qualified. Preserve native versus emulated execution labels and avoid performance comparisons between them.

Acceptance records, releases and operational limits

Proposed acceptance entry points are `validation/phase1/acceptance.py`, `validation/phase2/acceptance.py` and `validation/phase3/acceptance.py`, each with `--output` requiring a fresh location. These commands will be implemented by P1-06, P2-06 and P3-07; they are not runnable at this planning revision. Every runner records implementation/schema revisions, fixture and input digests, host/worker capabilities, commands, return codes, browser/replay artifacts, before/after preservation inventories, coverage/discrepancy IDs and reviewer disposition. Publish compact evidence; keep private state and large retained runtimes under documented separate retention.

Release gate	Required evidence and approval
G1 - native browsing subset	P1-01 through P1-06 pass; durable import/count/history/evidence results, real browser navigation, backup/restore and assigned local service ownership; M/P record the declared view scope
G2 - local Docker backend	G1 plus P2-01 through P2-06 pass; end-to-end submission, fault/retry/cleanup evidence and independent archive-only replay after deletion/relocation; M/P record target/prerequisite scope
G3 - selected native campaign	G2 plus P3-01 through P3-07 pass; adapter/adoption/shadow comparisons, reviewed discrepancy dispositions, one-writer transactions and tested routing rollback; M/P approve only the named campaigns

An unavailable historical campaign archive limits historical comparison; use the measured corpus and record that limitation. A missing runtime or required dependency limits replay, not inspection of retained bytes. Unavailable GPU/remote/vendor/organization infrastructure blocks the affected qualification rather than the independent local CPU implementation. P signs review only through a recorded human decision; automated test counts never set approval fields.

Phase 1 read-only views intentionally precede production mutations. Phase 2 dispatch introduces resource/state effects only after preflight and the established native store. Phase 3 changes verdict/discovery/enrollment behavior only after preserved failure replay is proven. This ordering avoids a new dashboard becoming an unverified replacement for the very debugging and adoption workflows the modernization must retain.

Evidence references and next action

Use the immutable Phase 0 proposal together with the local readiness report, dashboard ledger, triage acceptance scenarios, adoption inventory, repository authority map and runtime inventory. Retain the original evidence and status fields when producing new implementation results.

The first implementation change is P1-01 after G00 records the scope/owners/review disposition. Prepare P1-02 fixtures and the P1-04 browser acceptance script specification in parallel against that contract; start product data migration only after the contract decision is reviewed. Treat every milestone as a separate reviewable pull request with its own evidence, rather than bundling the entire three-phase implementation into one change.